Exercice 1 — Analyse de texte (Algorithmique + chaînes)

Énoncé

Écrire un programme Python qui :

1. Demande à l'utilisateur une phrase.
2. Affiche :
   - le nombre total de caractères (hors espaces),
   - le nombre de mots,
   - le mot le plus long,
   - le nombre de voyelles.
3. Affiche ensuite un dictionnaire contenant la fréquence de chaque mot.

Exemple :

```
{'python': 2, 'est': 1, 'super': 1}
```

Exercice 2 — Gestion d'une liste de nombres (Structures + logique)

Énoncé

Créer un programme qui :

1. Demande à l'utilisateur de saisir une liste de nombres.
2. Propose un menu :

- Trier la liste
- Afficher les nombres pairs
- Afficher les nombres impairs
- Calculer la moyenne
- Trouver le maximum et minimum
- Quitter

Chaque action doit être implémentée dans une fonction séparée.

Exercice 3 — Validation de mot de passe (conditions + regex)

Énoncé

Écrire une fonction :

```
validate_password(password)
```

Le mot de passe doit :

- contenir au moins 8 caractères
- contenir une majuscule
- contenir un chiffre
- contenir un caractère spécial

Retourner True ou False.

Exercice 4 — Lecture / écriture fichier

Énoncé

Créer un programme qui :

1. Lit un fichier `students.txt`
2. Chaque ligne contient :

```
nom;prenom;note
```

3. Charger les données dans une structure Python.
4. Calculer la moyenne générale.
5. Générer un fichier `result.txt` contenant :

```
Nom Prenom - Moyenne - Admis/Refusé
```

Exercice 5 — Mini système de gestion d'étudiants (POO)

Énoncé

Créer une application console permettant de gérer des étudiants.

Classe `Student`

Attributs :

- id
- nom
- prenom
- notes (liste)

Méthodes :

- add_note()
- average()
- is_admitted()

Classe `StudentManager`

Contient une liste de Student.

Méthodes :

- add_student()
- remove_student(id)
- find_student(id)
- list_students()
- best_student()

Créer un menu :

1. Ajouter étudiant
2. Ajouter note
3. Afficher tous
4. Meilleur étudiant
5. Quitter

Exercice 6 — Simulation bancaire (POO + exceptions)

Énoncé

Créer :

1. Classe `BankAccount`

Créer une classe `BankAccount` avec :

Attributs

- `owner`
- `balance`

Méthodes

- `deposit(amount)` : ajoute de l'argent au compte
- `withdraw(amount)` : retire de l'argent du compte

Le retrait ne doit être possible que si le solde est suffisant.

---

2. Gestion des erreurs

Créer une exception personnalisée :

```
InsufficientFundsError
```

Cette exception doit être levée quand on essaie de retirer plus d'argent que le solde disponible.

---

3. Programme principal

Dans le programme :

1. Créer au moins **trois comptes bancaires**
2. Effectuer plusieurs opérations :
   - dépôts
   - retraits
   - transferts entre comptes
3. Utiliser `try / except` pour gérer les erreurs

Un transfert consiste à :

- retirer l'argent du compte source
- ajouter ce même montant au compte destination

Exercice 7 — Gestion de commandes (POO + logique métier)

1. Classe `Produit`

Créer une classe `Produit` avec :

Attributs

- `id`
- `nom`
- `prix`

---

2. Classe `Commande`

Créer une classe `Commande` avec :

Attributs

- `produits` : liste de produits
- `status` : état de la commande (`pending`, `validated`, `cancelled`)

Méthodes

- `add_product(produit)` : ajoute un produit à la commande
- `total()` : calcule le montant total de la commande
- `validate()` : valide la commande (change le statut)

---

3. Classe `OrderManager`

Créer une classe `OrderManager` chargée de gérer toutes les commandes.

Elle doit proposer les méthodes :

- `create_order()` : créer une nouvelle commande
- `cancel_order(order_id)` : annuler une commande
- `list_orders()` : afficher toutes les commandes

---

4. Programme principal

Dans le programme principal :

1. Créer plusieurs produits
2. Créer une ou plusieurs commandes
3. Ajouter des produits aux commandes
4. Valider ou annuler certaines commandes
5. Afficher la liste des commandes avec leur total

Exercice 8 — Sauvegarde JSON

Énoncé

Ajouter à l'exercice précédent :

- sauvegarde des commandes dans `orders.json`
- chargement au démarrage

Exercice 9 — Héritage

Énoncé

Créer :

Classe abstraite `Employee`

- name
- base_salary

Méthode abstraite :

`calculate_salary()`

Classes :

- Developer
- Manager

Chaque classe calcule salaire différemment.

Créer une liste mixte.

Calculer masse salariale.

Exercice 10 — Mini projet final

Énoncé

Créer une application console de gestion de tâches :

Task

- id

- title
- done

TaskManager

- add_task()
- delete_task()
- complete_task()
- list_tasks()

Stocker dans fichier JSON.

Interface via menu.